Laboratorio 6 — Hoja de trabajo

Composer, herencia, interfaces, traits y enums

Nombre: Ronald Salvatierra Flores **Fecha:** 14 / 10 / 2026

Primero: `composer dump-autoload` (crea vendor/autoload.php). Luego `php demo.php` y `php verificar.php` (14 pruebas). Todo se ejecutó en Docker con `docker compose exec php …`.

Parte A — Composer y PSR-4

1. Ejecuta php verificar.php antes de composer dump-autoload. ¿Qué dice?

```
❌ Falta vendor/autoload.php. Primero ejecuta:  composer dump-autoload
```

Se detiene de inmediato y no corre ninguna prueba. Sin vendor/autoload.php, PHP no sabe dónde buscar las clases del proyecto.

2. Abre vendor/composer/autoload_psr4.php. ¿Qué relación hay entre Apicultores\ y src/?

```
return array(
    'Apicultores\\' => array($baseDir . '/src'),
);
```

El prefijo de espacio de nombres Apicultores\ corresponde a la carpeta src/. Lo que sigue al prefijo se convierte en subcarpetas: Apicultores\Modelo\Producto → src/Modelo/Producto.php.

3. Según PSR-4, ¿en qué archivo debe estar Apicultores\Modelo\Servicio? ¿Y Apicultores\Catalogo?

Apicultores\Modelo\Servicio → src/Modelo/Servicio.php. Apicultores\Catalogo → src/Catalogo.php. Cada clase va en su propio archivo y el archivo se llama igual que la clase.

4. Prueba el comando con Productos (plural). Copia el error. ¿Por qué ocurre?

```
PHP Fatal error:  Uncaught Error: Class "Apicultores\Modelo\Productos" not found in Command line code:1
```

La clase se llama Producto (singular). El autoload busca src/Modelo/Productos.php, ese archivo no existe y PHP responde que la clase no se encontró. Una letra de diferencia en el nombre rompe la carga.

5. ¿Para qué sirve "files": ["src/funciones.php"] en composer.json? ¿Por qué no se usa PSR-4 para ese archivo?

Le indica a Composer que cargue ese archivo siempre, apenas se incluye vendor/autoload.php. Ahí están las funciones globales e() y bs(). PSR-4 solo asocia nombres de clase con archivos y los carga cuando se usa la clase; una función suelta no es una clase y nadie la "pide" al autoload.

6. TODO 6 (public/api.php): reemplaza los require manuales por el autoload. ¿Qué problema tendrían los require con 40 clases?

Hecho: los 7 require se reemplazaron por una sola línea, require __DIR__ . '/../vendor/autoload.php';, seguida de use Apicultores\Catalogo;. Con 40 clases habría 40 require: hay que respetar el orden (interfaz y padre antes que el hijo), las rutas son frágiles, cada clase nueva obliga a editar todas las páginas y olvidar una produce un error fatal. El autoload carga cada clase solo cuando se usa.

Parte B — Enum (TODO 1 en src/Modelo/Categoria.php)

1. Completa descripcion() con match ($this).

```
public function descripcion(): string
{
    return match ($this) {
        self::Miel => 'Miel pura de la Chiquitanía',
        self::Derivados => 'Polen, propóleo y más',
```

```
        self::Cera => 'Cera de abeja',
    };
}
```

2. ¿Qué devuelven Categoria::from('Hongos') y Categoria::tryFrom('Hongos')? ¿Cuándo usarías cada uno?

```
from('Hongos')    → ValueError: "Hongos" is not a valid backing value for enum Apicultores\
Modelo\Categoria
tryFrom('Hongos') → NULL
```

Usaría from() cuando el valor debe ser válido sí o sí, por ejemplo al leer catalogo.json: si no lo es, es un error y conviene que falle. Usaría tryFrom() cuando el valor viene de afuera (un formulario o la URL) y quiero manejar el caso inválido con un if o con ?? sin lanzar excepciones.

3. ¿Qué ventaja tiene el enum frente a la constante CATEGORIAS del lab 3?

La constante es solo un arreglo de textos: cualquier string, incluso mal escrito, pasa sin queja y no tiene comportamiento. El enum es un tipo propio con solo tres valores posibles: Producto exige una Categoria y PHP rechaza cualquier otra cosa. Además puede tener métodos (descripcion(), slug()) y trae from(), tryFrom() y cases().

Parte C — Herencia y clase abstracta (TODO 2 y 3 en src/Modelo/Producto.php)

1. ¿Qué hereda Producto de Articulo? ¿Qué agrega?

Hereda codigo, nombre y precio (protected) con sus getters, la validación del código y del precio, precioFormateado(), el trait ConBolivianos y los contratos Vendible y JsonSerializable. Agrega stock, categoria y socio; tipo() y detalle(); el precio mayorista en precioVenta(); estaDisponible(), vender(), valorEnStock() y las constantes DESCUENTO_MAYORISTA y MINIMO_MAYORISTA.

2. ¿Qué pasa con new Articulo('ABC-10', 'x', 1.0) (paso 7 de la demo)? ¿Por qué tiene sentido?

```
Cannot instantiate abstract class Apicultores\Modelo\Articulo
```

Tiene sentido porque un "artículo" a secas no existe en la asociación: siempre es un producto físico o un servicio. La clase abstracta reúne lo común y obliga a las hijas a definir tipo() y detalle(). Si se pudiera instanciar habría objetos incompletos.

3. Completa el precio mayorista (TODO 2) y jsonSerialize con parent:: (TODO 3). Antes del TODO 3, ¿qué campos faltaban en /api.php?

```
public function precioVenta(int $cantidad): float
{
    $total = $this->precio * $cantidad;
    if ($cantidad >= self::MINIMO_MAYORISTA) {
        $total *= 1 - self::DESCUENTO_MAYORISTA / 100;
    }
    return round($total, 2);
}

public function jsonSerialize(): array
{
    return parent::jsonSerialize() + [
        'categoria' => $this->categoria->value,
        'stock' => $this->stock,
        'socio' => $this->socio,
    ];
}
```

Desde 12 unidades se aplica 10 % de descuento: 12 × 45 = 540 → 486. Antes del TODO 3, el JSON de un producto solo traía categoria, stock y socio; faltaban tipo, codigo, nombre y precio, que prepara Articulo. Con parent::jsonSerialize() + [...] se conserva lo del padre y se añade lo propio.

4. ¿Por qué parent::__construct(...) va al inicio del constructor de Producto?

Porque el constructor del padre asigna y valida las propiedades heredadas (codigo, nombre, precio). Primero se construye la parte "Articulo" del objeto, y si el padre rechaza un dato (código inválido, precio ≤ 0) la excepción se lanza antes de seguir con lo propio de Producto, como el stock. PHP no llama al constructor del padre automáticamente.

Parte D — Interfaz y polimorfismo (TODO 4 y 5)

1. ¿Qué exige la interfaz Vendible? ¿Qué clases la cumplen?

Exige cinco métodos: codigo(), nombre(), precioVenta(int), estaDisponible() y vender(int). La cumplen Producto y Servicio, a través de la clase abstracta Articulo, que declara implements Vendible.

2. Completa Servicio::estaDisponible() (TODO 4) y Catalogo::servicios() (TODO 5).

```
// Servicio.php: un servicio no tiene stock que se agote
public function estaDisponible(): bool
{
    return true;
}

// Catalogo.php
public function servicios(): array
{
    return array_values(array_filter(
        $this->articulos,
        fn(Vendible $a): bool => $a instanceof Servicio
    ));
}
```

3. Paso 4 de la demo: ¿por qué 12 unidades de miel cuestan 486 pero 12 personas en el taller cuestan 2400? ¿Qué método se ejecuta en cada caso y quién lo decide?

En la miel se ejecuta Producto::precioVenta(), que aplica 10 % desde 12 unidades: 540 − 10 % = 486. En el taller se ejecuta Servicio::precioVenta(), que no tiene descuento: 12 × 200 = 2400. Lo decide el objeto real en tiempo de ejecución: $miel es un Producto y $taller es un Servicio, y PHP ejecuta la versión de su clase. El código que llama (por ejemplo Catalogo::totalPedido) solo conoce la interfaz Vendible. Eso es polimorfismo.

4. Catalogo::agregar(Vendible $articulo): ¿qué pasa si le pasas un new DateTime()?

```
TypeError: Apicultores\Catalogo::agregar(): Argument #1 ($articulo) must be of type
Apicultores\Contratos\Vendible, DateTime given
```

DateTime no implementa Vendible, así que PHP lo rechaza. Usar la interfaz como tipo del parámetro actúa como filtro: solo entra lo que cumple el contrato.

Parte E — Trait

1. ¿Qué aporta ConBolivianos a Articulo? ¿Por qué enBolivianos() es protected?

Aporta el método enBolivianos(float), que formatea un monto como "Bs 45,00"; Articulo lo usa en precioFormateado(). Es protected porque es una ayuda interna de la clase y de sus hijas, no parte de lo que se ofrece al exterior: desde afuera se usa precioFormateado().

2. ¿En qué se diferencia un trait de una clase padre?

Una clase padre expresa una relación "es un" (Producto es un Articulo), se puede tener solo una y define un tipo que sirve para polimorfismo (instanceof). Un trait es código reutilizable que se "copia" dentro de la clase que lo usa: no es un tipo, no se puede instanciar, una clase puede usar varios y sirve para compartir comportamiento entre clases que no están emparentadas.

Parte F — Verificación

php -l en los cinco archivos modificados y php verificar.php: 14 de 14 comprobaciones.

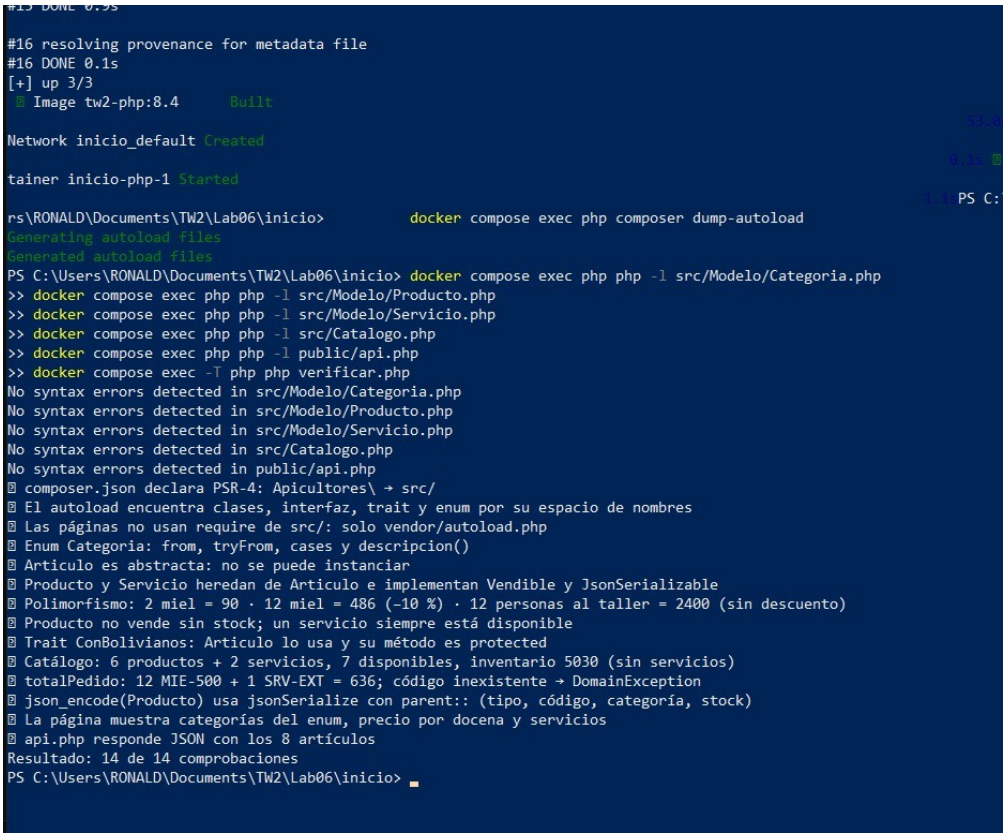


Figura 1. php -l sin errores y verificador con 14 de 14 comprobaciones.

Salida de php demo.php:

```
1) Nombre completo de la clase:  Apicultores\Modelo\Producto
2) Enum:                         Miel pura de la Chiquitanía | tryFrom("Hongos") → NULL
3) Herencia e interfaz:          true true false
4) Polimorfismo precioVenta:     2 miel → Bs 90,00 · 12 miel → Bs 486,00 · 12 taller → Bs 2.400,00
5) detalle():                    32 en stock | por persona
6) Trait:                        Apicultores\Soporte\ConBolivianos → Bs 45,00
7) new Articulo(...):            Cannot instantiate abstract class Apicultores\Modelo\Articulo
8) Pedido mixto:                 12 MIE-500 + 1 SRV-EXT → Bs 636,00
9) json_encode:                  {"tipo":"Servicio","codigo":"SRV-TAL","nombre":"Taller de iniciación
apícola","precio":200.0,"unidad":"persona"}
10) Inventario (sin servicios):  Bs 5.030,00
```

Páginas en el navegador:



Figura 2. Página principal: 8 artículos, 7 disponibles, inventario Bs 5.030,00, precio por docena y servicios.

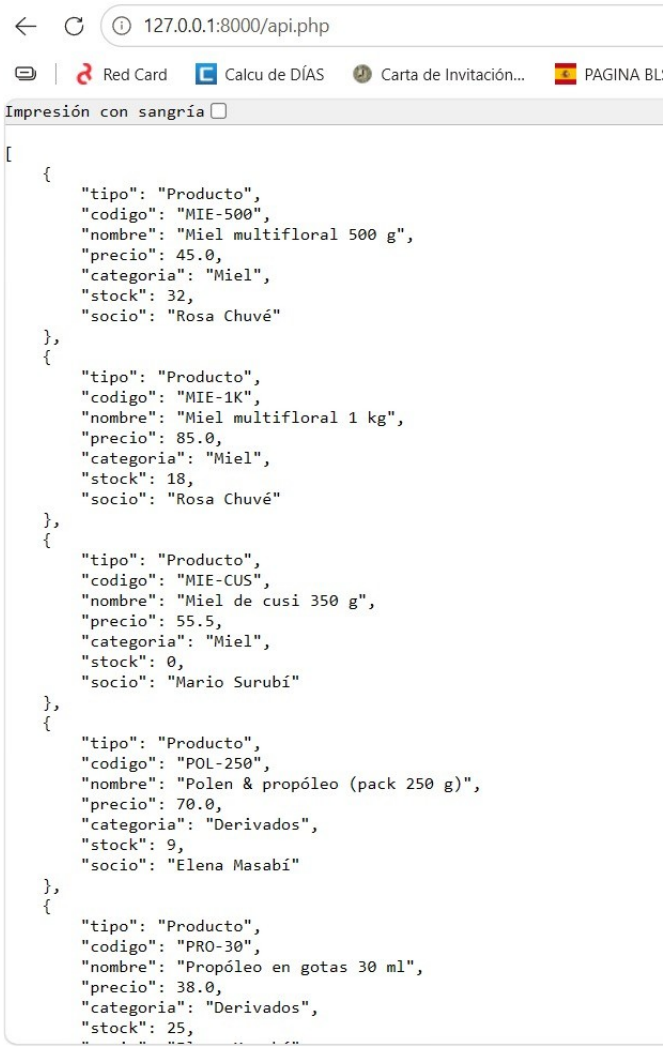


Figura 3. api.php: JSON con los campos heredados (tipo, codigo, nombre, precio) y los propios.

La página muestra 8 artículos, 7 disponibles, inventario Bs 5.030,00 (sin servicios), la docena de miel a Bs 486,00 y los dos servicios. La API entrega los 8 artículos en JSON.

Reflexión — Diagrama de clases

Relaciones: Producto y Servicio heredan de Articulo; Articulo implementa Vendible y JsonSerializable; Articulo usa el trait ConBolivianos; Producto tiene una Categoria (enum); Catalogo contiene objetos Vendible (0..*).

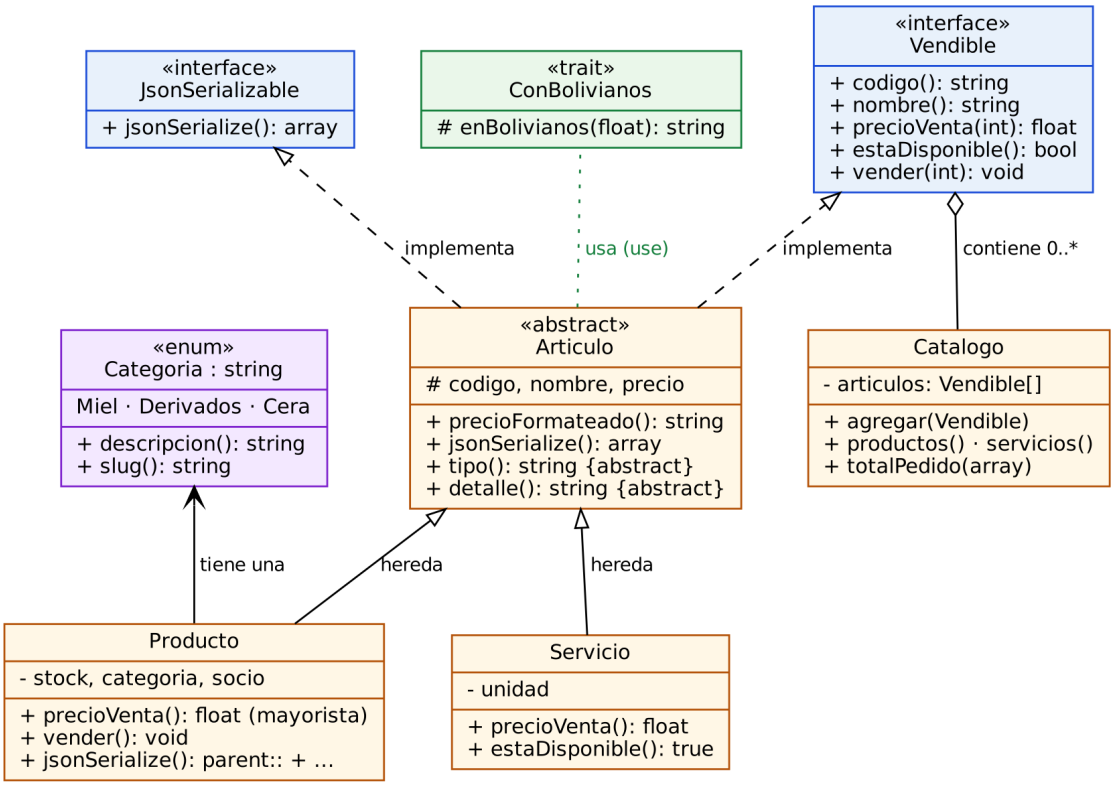


Figura 4. Diagrama de clases del catálogo.

Anexo — Archivos completados

Archivos de la carpeta inicio con los TODO resueltos. El resto de los archivos (Articulo, Vendible, ConBolivianos, funciones.php, index.php, composer.json, demo.php y verificar.php) no se modificaron.

src/Modelo/Categoria.php

```
<?php
// src/Modelo/Categoria.php — Enum respaldado por string: SOLO existen estas tres categorías.
// Reemplaza a la constante CATEGORIAS del lab 3 y a la clase Categoria del lab 5.
declare(strict_types=1);

namespace Apicultores\Modelo;

enum Categoria: string
{
    case Miel = 'Miel';
    case Derivados = 'Derivados';
    case Cera = 'Cera';

    public function descripcion(): string
    {
        // TODO 1: match ($this) devuelve la descripción de cada caso del enum.
        return match ($this) {
            self::Miel => 'Miel pura de la Chiquitanía',
            self::Derivados => 'Polen, propóleo y más',
            self::Cera => 'Cera de abeja',
        };
    }

    public function slug(): string
    {
        return strtolower($this->value);
    }
}
```

src/Modelo/Producto.php

```
<?php
// src/Modelo/Producto.php — Un producto físico: HEREDA de Articulo y agrega stock, categoría y socio.
declare(strict_types=1);

namespace Apicultores\Modelo;

use DomainException;
use InvalidArgumentException;

final class Producto extends Articulo
{
    public const DESCUENTO_MAYORISTA = 10;   // % desde 12 unidades
    public const MINIMO_MAYORISTA = 12;

    private int $stock;

    public function __construct(
        string $codigo,
        string $nombre,
```

```
        float $precio,
        int $stock,
        private readonly Categoria $categoria,
        private readonly string $socio = '',
    ) {
        parent::__construct($codigo, $nombre, $precio);   // primero, lo que valida el padre
        if ($stock < 0) {
            throw new InvalidArgumentException('El stock no puede ser negativo.');
        }
        $this->stock = $stock;
    }

    public function stock(): int { return $this->stock; }
    public function categoria(): Categoria { return $this->categoria; }
    public function socio(): string { return $this->socio; }

    public function tipo(): string
    {
        return 'Producto';
    }

    public function detalle(): string
    {
        return $this->stock > 0 ? "{$this->stock} en stock" : 'Agotado';
    }

    /** Precio mayorista: 10 % de descuento desde 12 unidades. */
    public function precioVenta(int $cantidad): float
    {
        // TODO 2: desde MINIMO_MAYORISTA unidades se aplica DESCUENTO_MAYORISTA % (12 miel de 45 → 486)
        $total = $this->precio * $cantidad;
        if ($cantidad >= self::MINIMO_MAYORISTA) {
            $total *= 1 - self::DESCUENTO_MAYORISTA / 100;
        }
        return round($total, 2);
    }

    public function estaDisponible(): bool
    {
        return $this->stock > 0;
    }

    public function vender(int $cantidad): void
    {
        if ($cantidad <= 0) {
            throw new InvalidArgumentException('La cantidad debe ser positiva.');
        }
        if ($cantidad > $this->stock) {
            throw new DomainException("Stock insuficiente de {$this->codigo}: hay {$this->stock}, se
pidieron $cantidad.");
        }
        $this->stock -= $cantidad;
    }

    public function valorEnStock(): float
    {
```

```
        return $this->precio * $this->stock;
    }

    public function jsonSerialize(): array
    {
        // TODO 3: reutiliza lo que arma el padre (tipo, codigo, nombre, precio) y AGREGA categoria,
stock y socio.
        return parent::jsonSerialize() + [
            'categoria' => $this->categoria->value,
            'stock' => $this->stock,
            'socio' => $this->socio,
        ];
    }
}
```

src/Modelo/Servicio.php

```
<?php
// src/Modelo/Servicio.php — Un servicio (alquiler, taller): hereda de Articulo pero NO tiene stock.
declare(strict_types=1);

namespace Apicultores\Modelo;

use InvalidArgumentException;

final class Servicio extends Articulo
{
    public function __construct(
        string $codigo,
        string $nombre,
        float $precio,
        private readonly string $unidad = 'día',
    ) {
        parent::__construct($codigo, $nombre, $precio);
    }

    public function unidad(): string { return $this->unidad; }

    public function tipo(): string
    {
        return 'Servicio';
    }

    public function detalle(): string
    {
        return "por {$this->unidad}";
    }

    /** Sin descuento mayorista: se cobra por día o por persona. */
    public function precioVenta(int $cantidad): float
    {
        return round($this->precio * $cantidad, 2);
    }

    public function estaDisponible(): bool
    {
        // TODO 4: un servicio no tiene stock que se agote, así que siempre está disponible.
```

```
        return true;
    }

    public function vender(int $cantidad): void
    {
        if ($cantidad <= 0) {
            throw new InvalidArgumentException('La cantidad debe ser positiva.');
        }
        // no hay stock que descontar
    }

    public function jsonSerialize(): array
    {
        return parent::jsonSerialize() + ['unidad' => $this->unidad];
    }
}
```

src/Catalogo.php

```
<?php
// src/Catalogo.php — Trabaja con la INTERFAZ Vendible: no le importa si es producto o servicio
(polimorfismo).
declare(strict_types=1);

namespace Apicultores;

use Apicultores\Contratos\Vendible;
use Apicultores\Modelo\Categoria;
use Apicultores\Modelo\Producto;
use Apicultores\Modelo\Servicio;
use DomainException;

final class Catalogo
{
    /** @var array<string, Vendible> */
    private array $articulos = [];

    public static function desdeArchivo(string $ruta): self
    {
        $catalogo = new self();
        foreach (json_decode((string) file_get_contents($ruta), true, 512, JSON_THROW_ON_ERROR) as $f) {
            $catalogo->agregar(match ($f['tipo']) {
                'producto' => new Producto($f['codigo'], $f['nombre'], (float) $f['precio'], (int)
$f['stock'],
                    Categoria::from($f['categoria']), $f['socio'] ?? ''),
                'servicio' => new Servicio($f['codigo'], $f['nombre'], (float) $f['precio'],
$f['unidad'] ?? 'día'),
            });
        }
        return $catalogo;
    }

    public function agregar(Vendible $articulo): void
    {
        if (isset($this->articulos[$articulo->codigo()])) {
            throw new DomainException("Ya existe el código {$articulo->codigo()}.");
        }
```

```
        $this->articulos[$articulo->codigo()] = $articulo;
    }

    public function buscar(string $codigo): ?Vendible
    {
        return $this->articulos[strtoupper(trim($codigo))] ?? null;
    }

    /** @return Vendible[] */
    public function todos(): array
    {
        return array_values($this->articulos);
    }

    /** @return Producto[] */
    public function productos(?Categoria $categoria = null): array
    {
        return array_values(array_filter(
            $this->articulos,
            fn(Vendible $a): bool => $a instanceof Producto && ($categoria === null || $a->categoria()
=== $categoria)
        ));
    }

    /** @return Servicio[] */
    public function servicios(): array
    {
        // TODO 5: igual que productos(), pero filtrando con instanceof Servicio
        return array_values(array_filter($this->articulos, fn(Vendible $a): bool => $a instanceof
Servicio));
    }

    /** @return Vendible[] */
    public function disponibles(): array
    {
        return array_values(array_filter($this->articulos, fn(Vendible $a): bool => $a-
>estaDisponible()));
    }

    /** Solo los productos tienen inventario: los servicios no suman. */
    public function valorInventario(): float
    {
        return array_sum(array_map(fn(Producto $p): float => $p->valorEnStock(), $this->productos()));
    }

    /** Total de un pedido: cada artículo calcula SU precio (polimorfismo). @param array<string,int>
$pedido */
    public function totalPedido(array $pedido): float
    {
        $total = 0.0;
        foreach ($pedido as $codigo => $cantidad) {
            $articulo = $this->buscar($codigo) ?? throw new DomainException("No existe el código
$codigo.");
            $total += $articulo->precioVenta($cantidad);
        }
        return round($total, 2);
```

```
    }
}
```

public/api.php

```
<?php
// public/api.php — El catálogo en JSON (anticipo de la API REST de la clase 13): json_encode usa
jsonSerialize().
declare(strict_types=1);
// TODO 6: UNA sola línea, el autoload de Composer, en lugar de 7 require manuales (igual que
index.php).
require __DIR__ . '/../vendor/autoload.php';

use Apicultores\Catalogo;

$catalogo = Catalogo::desdeArchivo(__DIR__ . '/../datos/catalogo.json');
header('Content-Type: application/json; charset=utf-8');
echo json_encode($catalogo->todos(), JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE |
JSON_PRESERVE_ZERO_FRACTION | JSON_THROW_ON_ERROR);
```